

# **Proyecto Programado II: Resolución de Laberintos**

Instituto Tecnológico de Costa Rica

Sede Interuniversitaria de Alajuela

Santiago Escamilla Sanabria

29/05/2026

## Resumen Ejecutivo

Este proyecto es un programa en Python 3 que resuelve laberintos dados como matrices de texto. La idea es encontrar un camino desde el punto de inicio hasta el punto final usando backtracking con una pila, retrocediendo cada vez que se llega a un callejón sin salida.

El programa lee el laberinto desde la entrada estandar, encuentra el camino si existe y lo marca con asteriscos. Si no hay ningun camino posible simplemente dice que no existe solución. Lo probé con 5 casos distintos y todos funcionaron como esperaba.

## Introducción

Resolver un laberinto es básicamente encontrar una ruta entre dos puntos evitando obstáculos, y aunque suena simple es un problema muy útil para entender cómo funcionan la búsqueda exhaustiva y el backtracking en programación.

El objetivo de este proyecto es implementar ese algoritmo en Python, recibiendo el laberinto como una matriz de caracteres y devolviendo el camino encontrado marcado con asteriscos, o indicando que no existe solución si ese es el caso.

## Marco Teórico

### Matrices

Una matriz es una estructura de datos en dos dimensiones, organizada en filas y columnas. En este proyecto el laberinto es una lista de listas en Python, donde cada celda tiene un caracter que indica si es espacio libre, pared, inicio o fin.

### Pilas

Una pila es una estructura tipo LIFO, o sea, el último en entrar es el primero en salir. En Python se puede simular con una lista normal usando `append` para agregar elementos y `pop` para sacarlos. Aquí la use para guardar las celdas que faltan por explorar junto con el camino recorrido hasta ese punto.

### Backtracking

El backtracking es una tecnica que consiste en explorar todas las posibilidades de forma ordenada, y cuando un camino no lleva a ningún lado simplemente se abandona y se vuelve atrás. En el caso del laberinto ese retroceso ocurre de forma automática cuando se hace `pop` de la pila, ya que eso nos devuelve a la celda anterior que todavía tenía opciones sin explorar.

### Búsqueda Exhaustiva

La búsqueda exhaustiva recorre todos los caminos posibles hasta encontrar la solución o

confirmar que no existe ninguna. Combinada con la matriz de visitados, garantiza que ninguna celda se procese más de una vez, lo que da una complejidad de  $O(F \times C)$  donde F es el número de filas y C el de columnas.

## Descripción de la Solución

El programa lo dividí en funciones pequeñas donde cada una hace una sola cosa. Hay funciones para leer el laberinto, buscar el inicio, verificar si una celda es valida, resolver el laberinto con backtracking, marcar el camino encontrado e imprimirlo. La función main coordina todo el flujo.

El algoritmo principal crea una matriz de visitados para no repetir celdas, luego mete el punto de inicio en la pila y empieza a explorar. En cada iteración saca la celda del tope de la pila, revisa si es el destino, y si no lo es agrega a la pila todos los vecinos validos con el camino actualizado. Si la pila se vacia sin haber encontrado el destino, significa que no hay solución.

Listing 1: Función principal de backtracking

```
1 def resolver(mapa, filas, cols, inicio):
2     # backtracking con pila, explora los 4 movimientos posibles
3     dirs = [(-1,0), (1,0), (0,-1), (0,1)] # arriba, abajo, izq, der
4     visitados = [[False] * cols for _ in range(filas)]
5
6     # cada elemento: (fila, col, camino recorrido)
7     pila = [(inicio[0], inicio[1], [inicio])]
8     visitados[inicio[0]][inicio[1]] = True
9
10    while pila:
11        f, c, camino = pila.pop() # backtracking implicito
12
13        if mapa[f][c] == 'E':
14            return camino # encontramos el fin
15
16        for df, dc in dirs:
```

```
17         nf, nc = f + df, c + dc
18         if es_valida(nf, nc, filas, cols, mapa, visitados):
19             visitados[nf][nc] = True
20             pila.append((nf, nc, camino + [(nf, nc)]))
21
22     return None # pila vacia = no hay solucion
```

## Resultados de Pruebas

### Prueba 1 — Camino directo

Entrada:

3 5

S...E

#####

.....

Salida obtenida:

S\*\*\*E

#####

.....

El camino es directo en la primera fila y el algoritmo lo encontró sin necesidad de retroceder.

### Prueba 2 — Sin solución

Entrada:

3 3

S.#

###

#.E



Salida obtenida:

No existe solución

Las paredes bloquean completamente el paso entre S y E, la pila se vacia sin encontrar el destino.

### Prueba 3 — Camino con retrocesos

Entrada:

5 5

S.#..

.####

.....

####.

....E

Salida obtenida:

S.#..

\*####

\*\*\*\*\*

####\*

....E

Acá el algoritmo tuvo que explorar hacia abajo y serpentear por el laberinto, demostrando el retroceso cuando intentaba caminos sin salida.

## Prueba 4 — Laberinto grande

Entrada:

8 10

S.....

.#####.

.....

.#####.

.....

.#####.

.....

.....E

Salida obtenida:

S\*\*\*\*\*

.#####\*

\*\*\*\*\*

\*#####.

\*\*\*\*\*

.#####\*

.....\*

.....E

Sirve para verificar que el programa funciona bien con laberintos mas grandes y no se rompe.

## Prueba 5 — Múltiples caminos

Entrada:

4 6

S....E

.....

.....

.....

Salida obtenida:

S\*\*\*\*E

.....

.....

.....

Hay muchos caminos posibles en este caso. El algoritmo elige el camino directo por la primera fila dependiendo del orden en que la pila explora las direcciones.

## Conclusiones

- El backtracking con pila es una forma muy limpia de resolver este tipo de problemas porque el retroceso ocurre solo con el pop, sin necesidad de logica extra.
- La matriz de visitados es esencial para que el algoritmo no entre en ciclos y sea eficiente.
- El programa maneja bien todos los casos pedidos: con solución, sin solución, caminos largos, laberintos grandes y múltiples caminos posibles.
- La complejidad es  $O(F \times C)$  en el peor caso ya que cada celda se visita a lo sumo una vez.

## Aprendizajes

- Aprendí a implementar backtracking de forma iterativa con pila en vez de recursión, lo que evita problemas de desbordamiento en laberintos muy grandes.
- Reforcé el manejo de matrices en Python y la creación de estructuras auxiliares como la matriz de visitados.
- Me quedó mas claro cuando conviene usar DFS con pila versus BFS con cola, y por qué DFS es más natural para backtracking.
- Aprendí a diseñar casos de prueba que cubran todos los escenarios posibles, no solo el caso donde todo funciona bien.